Protocol-as-System: A Declarative Protocol Computing Paradigm for Auditable AI Generation

Cheung Tiehua

cheungtiehua@gmail.com

Abstract

As Large Language Models (LLMs) and agentic systems evolve from passive tools into action-generating components capable of planning, decision-making, and tool orchestration, artificial intelligence systems are increasingly deployed in complex, high-risk, and high-governance-requirement scenarios. However, current mainstream AI architectures are still largely built upon the assumption of "model-autonomous execution." Their governance and control mechanisms primarily rely on model internal alignment, reward optimization, or post-hoc monitoring, making it difficult to provide strong controllability, non-bypassability, and auditability at the system level 1.

This paper proposes a novel system-level AI architecture paradigm——Protocol-as-System (PaS) . In this paradigm, system behavior is no longer directly determined by executable code or model strategies, but is uniformly adjudicated by an explicit, non-bypassable, and auditable set of protocols. This paper specifically emphasizes: protocols in PaS are declarative protocols——protocols only declare the legitimate boundaries of the system state space, rather than describing any execution steps, processes, or algorithms. Artificial intelligence is formalized as a "state transition proposal generator," not an autonomous executor.

We formalize this paradigm as a Protocol-Constrained Interactive Transition System (PC-ITS) , and present a systematic instantiation consistent with this paradigm: Declarative Operational Closure (DOC) ——an AI interactive application system centered on structured protocol objects, meta-protocol generation, and adjudication. Theoretically, PaS/PC-ITS provides system-level guarantees of safety, non-bypassability, non-autonomy, and complete auditability; practically, the fixed format and explicit feedability of DOC enable protocols to be identified, mounted, replayed, and audited.

---

1. Introduction

1.1 From "Tool Output" to "State Proposal"

Early AI systems primarily existed as input-output mapping tools, where their outputs were typically regarded as suggestions or answers. With the enhancement of model capabilities, AI outputs increasingly manifest as executable plans, schedulable tasks, callable tool sequences, and even the orchestration structures of multi-agent collaboration. This means AI outputs in a system are closer to "proposals for changing the system state" rather than single-shot inference results.

However, current mainstream AI governance methods are mostly concentrated at the model layer or prompt layer, making it difficult to provide strong controllability, non-bypassability, and auditability at the system level 1.

1.2 Lack of System-Level Governance

Existing governance methods primarily focus on the model layer (alignment training, reward constraints) or the prompt layer (rule prompting, safety prompts, refusal strategies). For instance, some research builds trust through constructing cryptographically based verifiable audit trails 2, or utilizes knowledge graphs to enhance the traceability of decision processes 3. Industry practices tend towards building layered compliance audit frameworks covering the entire lifecycle 4. These methods share a common limitation: they influence "what the model says," but struggle to enforce "what the system allows to happen." When a system lacks an adjudication layer that is independent of the model, computable, and non-bypassable, the adoption of model outputs may directly lead to state violations or consequences that are not attributable.

1.3 Research Question and Contributions

This paper focuses on the following question: Can we construct an AI system where system behavior does not depend on model self-discipline, but is instead constrained by an external, explicit, auditable, and non-bypassable protocol system?

Main contributions include:

1. Proposing the PaS Paradigm: Shifting system control from inside the model to an external protocol layer.
2. Providing the PC-ITS Formalization: Modeling AI behavior as "state transition proposals" and protocols as "declarations and adjudication of state legitimacy."
3. Clearly Distinguishing Declarative Protocols from Executable Rule Systems: Providing definitions that preclude misinterpretation.
4. Presenting a Systematic Instantiation DOC: Demonstrating how PaS achieves auditability and governance in engineering implementation through fixed format, explicit feedability, and replayability.

---

2. Protocol-as-System (PaS) Paradigm

2.1 Core Proposition: Protocols are Declarative, Not Executable

The first core proposition of PaS is:

Protocols are not about "doing"; protocols only declare "what states are allowed to exist."

Therefore, protocols in PaS have the following strict meaning:

- Protocols do not describe execution steps (no processes, no algorithms, no scheduling).
- Protocols do not trigger execution (no side effects).
- Protocols do not undertake system evolution (system evolution occurs through external writing mechanisms).
- Protocols only provide: a declarative adjudication of "whether a state is reachable/writable."

This definition serves to structurally prevent PaS from being misinterpreted as a workflow engine, policy-as-code, or a "rule-driven execution system." In PaS, "execution/generation" belongs to external system mechanisms; the "protocol" solely provides a non-bypassable declaration of state boundaries.

Note: The protocols referred to in this paper are purely declarative protocols. Their semantics are limited to declaring the legitimacy of system states and do not contain any execution, scheduling, or process control semantics.

2.2 Architectural Paradigm Shift: From "Behavior Control" to "State Adjudication"

Traditional systems often attempt to control the behavior process: monitoring action sequences, limiting tool calls, auditing execution paths. However, in agentic systems, the behavior space is vast and difficult to exhaust; conversely, the state space boundary is often easier to define and audit.

PaS shifts the governance focus from the behavior process to the state boundary:

- AI can propose arbitrarily complex action plans.
- The system only cares about: whether the target state resulting from adopting the plan is declared legitimate by the protocol.

This transforms governance from "process supervision" to "boundary adjudication," thereby achieving stronger consistency and auditability.

2.3 Non-Autonomy: AI Can Only Propose, Does Not Possess Final Write Authority

Since protocols themselves do not execute, actual changes to the system state require external writing mechanisms. PaS therefore naturally satisfies non-autonomy:

- AI output is a proposal.
- The protocol adjudicates whether the state corresponding to the proposal is writable.
- The final write is completed by an external mechanism (which may include humans or trusted third parties).

2.4 Clarification: Protocol-as-System is Not an Execution System

When introducing the Protocol-as-System (PaS) paradigm, it is necessary to clearly distinguish it from several similar but fundamentally different system paradigms, to avoid misinterpreting the proposed declarative protocol system as an execution control system.

First, PaS is not a workflow engine.

The core objective of a workflow engine is to specify step sequences, execution conditions, and control flow transitions; its rules are directly involved in "how to execute tasks." In contrast, protocols in PaS do not describe any steps, processes, or control paths, nor do they prescribe how tasks should be completed. Protocols only declare the set of legitimate states within the system state space, without concern for which behaviors or paths led to those states.

Second, PaS is not policy-as-code or a rule-driven execution system.

In such systems, policy rules typically exist in executable form and perform real-time intervention or scheduling on behavior during runtime. Differently, protocols in PaS lack execution capability and produce no side effects; the protocol predicate merely provides a declarative adjudication on "whether the target state resulting from adopting a certain proposal is permitted." The protocol itself neither triggers actions nor modifies system state.

Third, PaS is not a variant of an operating system or virtual operating system.

The core function of an operating system lies in resource management, scheduling, and execution isolation, with its rules inherently coupled with execution mechanisms. PaS explicitly refuses to bind protocols with execution: the actual writing of system state always occurs outside the protocol, completed by external mechanisms. The protocol's role in the system is not as a "scheduler," but as a "non-bypassable source of legitimacy declarations."

Therefore, the core innovation of PaS lies not in proposing a new execution model, but in proposing a governance structure with declarative protocols as the system ontology. In this structure, the controllability of system behavior comes not from fine-grained monitoring of the execution process, but from the explicit declaration and non-bypassable adjudication of state space boundaries.

---

3. Formal Model: Protocol-Constrained Interactive Transition System (PC-ITS)

The formal foundation of the PC-ITS model is rooted in the theories and methods of formal verification 5. We formalize PaS as a 5-tuple:

$\Sigma = \langle S, A, P, \delta, \Omega \rangle$

Where:

- S: System State Space.
- A: Action Space (set of behavior/state transition proposals).
- P: Set of Protocol Predicates (non-bypassable adjudication rules).
- δ : Protocol-Constrained State Transition Function.
- Ω : External Sovereign Source, embodying non-autonomy.

3.1 State Space S: Explicit, Observable, Auditable

System state is defined as an explicit structured object, for example:

```
State = {
   mode: SystemMode,              // Running, Audit, Maintenance
   risk_level: RiskLevel,      // Low, Medium, High
   active_apps: Set[AppID], // Set of active applications
   audit_summary: AuditSummary // Audit summary
}
```

The system has no implicit states or unobservable variables.

3.2 Action Space A: Proposals, Not Execution

Each "action" is modeled as a proposal object, without carrying execution semantics:

```
ActionProposal = {
   agent_id: AgentID,
   action_type: ActionType,
   parameters: Dict[str, Any],
   confidence: Float,
   rationale: String
}
```

Key Point: The semantics of ActionProposal are not "execute an action," but "propose a change that might lead to a state change."

3.3 Protocol Predicates P: Purely Declarative Adjudication, No Side Effects

Protocols consist of a set of non-bypassable predicates:

```
ProtocolPredicate: (State, ActionProposal) → {ALLOW, DENY, ESCALATE}
```

Within the semantics of this paper, ProtocolPredicate must satisfy:

- Pure Function: Does not modify state, produces no side effects.
- Declarative Adjudication: Its output merely indicates "whether the target state is declared writable."
- Non-bypassability: All potential state writes must undergo adjudication by this predicate.

Here, ESCALATE indicates that the protocol cannot close the adjudication on the legitimacy of the target state with currently available information, requiring intervention from an external sovereign source Ω to complete the adjudication.

3.4 State Transition Function δ : Defining "Writes" as Auditable Events

The state transition function ensures all transitions undergo protocol adjudication:

```
                 ⎧
                 ⎪  s′          if P(s, a) = ALLOW
δ (s, a) =       ⎨  s           if P(s, a) = DENY
                 ⎪  δ_Ω (s, a) if P(s, a) = ESCALATE
                 ⎩
```

The critical implementation point of the declarative protocol lies here:

- P only returns the adjudication result.
- The actual writing behavior that causes state changes occurs within the external mechanism of δ (especially δ_Ω).

Therefore, the protocol does not execute the system; it only declares which states the system is permitted to enter.

---

4. Systematic Instantiation: Declarative Operational Closure (DOC)

This section presents a systematic instantiation consistent with the PaS/PC-ITS semantics, demonstrating how declarative protocols can be engineered into identifiable, mountable, replayable, and auditable structural objects. We emphasize: DOC is used here as "a representative example" ; its purpose is to demonstrate the feasibility of implementing PaS, not

to bind the paper to a specific product or system name.

4.1 Basic Definition of DOC: Protocol Objectification and Runnable Format

DOC elevates the "protocol" from scattered text or implicit rules to an identifiable structured object, ensuring parsability and simulatable execution through a fixed format.

This corresponds to the PaS requirement: protocols must be computable objects, not merely linguistic hints for the model.

4.2 Explicit Feedability and Replayability: Necessary Conditions for Auditability

DOC explicitly requires explicit feedability: single-file delivery, platform independence, replayability, complete text.

This provides the engineering foundation for PaS's auditability goal: protocol objects can be repeatedly input, re-adjudicated, and re-verified, enabling "auditable recomputation."

4.3 Legitimacy and Non-Repudiable Generation Chain (Optional Enhancement)

In scenarios with strong governance constraints, the "source of legitimacy" for protocols needs to be fixed; otherwise, arbitrarily forged protocols could lead to governance failure. DOC achieves this through "freezing the source of legitimacy," requiring legitimacy to originate solely from a certified generation chain and certified stamp, including self-destruction clauses to prevent boundary circumvention.

This aligns with PaS's "non-bypassability" requirement: protocols must not only declare boundaries but also be identifiable within the system as the "sole valid declaration of boundaries."

---

5. Theoretical Properties

- Theorem 1 (Safety): The system will never enter a state explicitly prohibited by the protocol.
  - Reasoning: Any potential write must pass through P; when the protocol declares the target state illegal, the write is denied.
- Theorem 2 (Non-bypassability): Any state change must undergo protocol adjudication.
  - Reasoning: δ is the sole entry point for state changes, and its definition mandatorily invokes P.
- Theorem 3 (Non-autonomy): Artificial intelligence does not possess the system's final write authority.
  - Reasoning: AI only generates proposals a $\in$ A. The final write is completed by the external mechanism of δ, determined by Ω when necessary.

- Theorem 4 (Complete Auditability): All system state changes can be fully audited and replayed.
  - Reasoning: State explication + Proposal objectification + Protocol adjudication results are recordable. If the implementation supports explicit feedability and replayability, audits can be recomputed.

---

6. Adversarial Scenario Evaluation and Analysis (Declarative Protocol Perspective)

- Scenario 1: Restricted Resource/Data Access
  - An AI proposes accessing sensitive resources. The system does not attempt to analyze the AI's "access process." Instead, it adjudicates whether the target state resulting from access (e.g., "sensitive data exposed") belongs to the set of states permitted by the protocol. If illegal -> DENY; if authorization required -> ESCALATE.
- Scenario 2: Multi-Application Resource Conflict
  - Multiple proposals compete for scarce resources simultaneously. The protocol declares that legitimate states must satisfy resource non-over-allocation, priority constraints, and fairness constraints. The system adjudicates whether the "state after resource allocation" satisfies the declarative constraints, rather than allowing the execution process to compete autonomously.
- Scenario 3: Escalation Storm (Excessive ESCALATE)
  - When a large number of proposals enter ESCALATE, external sovereign source Ω becomes a bottleneck. Declarative protocols can include legitimacy constraints on the "escalation frequency/queue state." For example, exceeding a threshold makes the target state illegal, thereby encoding system stability into the state boundaries themselves.

---

7. Discussion and Limitations

1. Protocol Completeness: Declarative protocols must cover all critical state boundaries; otherwise, gray areas remain.
2. Expressiveness and Maintainability: Stronger state constraints lead to higher maintenance costs.
3. Performance Overhead: Adjudication on every write introduces additional latency.
4. External Sovereign Bottleneck: ESCALATE requires high-quality Ω intervention; otherwise, throughput is limited.

These limitations reflect the fundamental cost of system governance: trading deterministic boundaries for controllability and auditability.

7.1 Key Differences from Related Work: A Declarative Protocol Perspective

Existing research on AI governance and safety can be broadly categorized into three types: model internal alignment methods, execution environment isolation methods, and rule or policy-based

control methods. While the PaS paradigm shares goals with these works, it fundamentally differs in the placement of system control authority. The 'Protocol-as-System' paradigm proposed in this study is fundamentally different from paths relying on external add-on tools 2, 3, 4. It achieves a migration of the trust foundation from external sources to an intrinsic adjudication process by positioning declarative protocols as the system core.

Compared to model internal alignment methods (e.g., reward modeling, safety fine-tuning), PaS does not attempt to constrain behavior by optimizing internal model parameters. Instead, it externalizes control authority to a protocol layer outside the model. In PaS, the model is not assumed to be a "trusted executor"; its output is merely viewed as a state transition proposal, and whether it is adopted depends entirely on external protocol adjudication.

Compared to execution environment isolation methods (e.g., sandboxing, Trusted Execution Environments, virtualization), PaS does not primarily rely on "limiting execution capability." Execution environment isolation focuses on "preventing damage even if execution goes wrong," whereas PaS focuses on "which system states are structurally disallowed from being written." These are orthogonal, but PaS advances the governance focus to the state legitimacy declaration layer.

Compared to rule or policy-driven systems, the key difference of PaS lies in its declarative, rather than imperative, protocol semantics. Rules in rule-driven systems typically directly influence the system's execution path. In contrast, protocols in PaS do not participate in execution; they exist solely as a non-bypassable declaration of state boundaries. This design renders protocols inherently auditable and recomputable, as their semantics do not depend on runtime behavior, but only on state mapping relationships.

In this sense, PaS is not an enhancement of existing control techniques, but a paradigm shift in system control: moving from "controlling how the system runs" to "declaring what states the system is permitted to become."

---

8. Conclusion

This paper proposes the "Protocol-as-System (PaS)" paradigm and formalizes it as PC-ITS. The most important clarification of this paper is: protocols in PaS are declarative protocols — — protocols do not execute the system, do not describe processes or algorithms, and only declare the legitimate boundaries of the system state space; AI can only generate state transition proposals, while the system's final write authority is jointly guaranteed by non-bypassable protocol adjudication and external sovereign mechanisms.

As a systematic instantiation, DOC demonstrates how declarative protocols can achieve identifiability, replayability, and auditability through fixed formats and explicit feedability, thereby providing system-level controllability and accountability traceability for

high-governance-requirement scenarios.

References

1

KOLLIPARA Y V P. Assured, Explainable, And Auditable AI For High-Stakes Decisions: A Survey Of Trustworthy Machine Learning In Mission-Critical SystemsJ. Journal of International Crisis and Risk Communication Research, 2025.

2

VERITASCHAIN. Empirical Study: Cryptographically Verifiable Audit Trails in Real-Time Algorithmic Trading: An Empirical Study Based on VCP v1.0; and Regulatory Analysis: Why Open Cryptographic Standards are Crucial for AI AuditabilityR, 2025.

3

KUMAR A, SHRESTHA Y R, VON KROGH G. Managing AI Bias Through Organizational Learning: The Role of Transparency and Accountability in Algorithmic AuditingC // Proceedings of the 56th Hawaii International Conference on System Sciences. 2023.

4

Alibaba Cloud. AI Full Lifecycle Compliance Audit GovernanceEB/OL. Alibaba Cloud Compliance Audit - Cloud Adoption Framework Documentation, 2025.

5

SELIGMAN E, SCHUBERT T, KUMAR M V A K. Formal Verification: An Essential Toolkit for Modern VLSI DesignM. Waltham: Elsevier Science, 2015.

协议即系统（Protocol-as-System）：一种用于可审计 AI 生成的声明式协议计算范式

Cheung Tiehua
cheungtiehua@gmail.com

摘要

随着大语言模型（Large Language Models, LLM）与智能体系统从被动工具逐步演化为具备规划、决策与工具编排能力的行动生成组件，人工智能系统正越来越多地被部署在复杂、高风险与高治理要求的场景中。然而，现有主流 AI 架构仍多建立在“模型自治执行”的假设之上，其治理与控制机制主要依赖模型内部对齐、奖励优化或事后监控，难以在系统层面提供强可控性、不可绕过性与可审计性[1]。

本文提出一种新的系统级人工智能架构范式——协议即系统（Protocol-as-System, PaS）。在该范式中，系统行为不再由可执行代码或模型策略直接决定，而是由一套显式、不可绕过、可审计的协议进行统一裁定。本文特别强调：协议在 PaS 中是声明式协议——协议只声明系统状态空间的合法性边界，而非描述任何执行步骤、流程或算法。人工智能被形式化为“状态变更提议生成器”，而非自主执行者。

我们将该范式形式化为一种协议约束交互式状态转移系统（Protocol-Constrained Interactive Transition System, PC-ITS），并给出一个与该范式一致的体系化实例：声明式运行闭包（Declarative Operational Closure, DOC）——一种以结构化协议对象为载体、以元协议生成与裁定为核心的 AI 交互式应用体系。理论上，PaS/PC-ITS 提供了安全性、不可绕过性、非自治性与完全可审计性的系统级保证；实践上，DOC 的固定形制与显性投喂底线使协议能够被识别、挂载、重放与审计。

1. 引言

1.1 从“工具输出”到“状态提议”

早期 AI 系统主要作为输入输出映射工具存在，其输出通常被视为建议或答案；随着模型能力提升，AI 输出越来越多地表现为可执行的计划、可调度的任务、可调用的工具序列，乃至多智能体协作的编排结构。这意味着 AI 输出在系统中更接近“对系统状态的变更提议”，而非单次推理结果。然而，当前主流的 AI 治理手段多集中于模型层或提示层，这些方法难以在系统层面提供强可控性、不可绕过性与可审计性[1]。

1.2 系统级治理缺失

现有治理手段多集中于模型层（对齐训练、奖励约束）或提示层（规则提示、安全提示、拒答策略）。例如，有研究通过构建基于密码学的可验证审计追踪来建立信任[2]，或利用知识图谱提升决策流程的可追溯性[3]。行业实践则倾向于构建覆盖全生命周期的分层合规审计框架[4]。这些手段存在共同局限：它们影响“模型怎么说”，但难以强制保证“系统允许发生什么”。当系统缺乏一个独立于模型的、可计算且不可绕过的裁定层时，模型输出被采纳后可能直接造成状态越界或不可追责的后果。

1.3 研究问题与贡献

本文聚焦以下问题：能否构建一种 AI 系统，使系统行为不依赖模型自律，而是受一套外在、显式、可审计、不可绕过的协议体系所约束？

主要贡献包括：

1. 提出 PaS 范式：将系统控制权从模型内部迁移至外部协议层；
2. 给出 PC-ITS 形式化：将 AI 行为建模为“状态变更提议”，将协议建模为“状态合法性声明与裁定”；
3. 明确区分声明式协议与执行式规则系统，给出不可误读的定义；
4. 给出体系化实例 AIC：展示 PaS 在工程实现中如何通过固定形制、显性投喂与可重放性实现审计与治理。

2. 协议即系统（PaS）范式

2.1 核心命题：协议是声明式的，不是执行式的

PaS 的第一核心命题是：

协议不是“做事”的；协议只声明“什么状态允许存在”。

因此，PaS 中的协议具有如下严格含义：

- 协议不描述执行步骤（无流程、无算法、无调度）
- 协议不触发执行（无副作用）
- 协议不承担系统演化（系统演化由外部写入机制发生）
- 协议仅提供：对“状态是否可达/可写入”的声明式裁定

这一定义用于从结构上避免 PaS 被误读为工作流引擎、策略即代码或“规则驱动执行系统”。在 PaS 中，“执行/生成”属于系统外部机制；“协议”仅提供对状态边界的不可绕过声明。

注：本文所称协议为纯声明式协议，其语义仅限于系统状态合法性声明，不包含任何执行、调度或流程控制语义。

2.2 架构范式转变：从“行为控制”到“状态裁定”

传统系统往往试图控制行为过程：监控动作序列、限制工具调用、审计执行路径。但在智能体系统中，行为空间巨大且难以穷尽；反之，状态空间边界通常更易被定义与审计。

PaS 将治理焦点从行为过程转移到状态边界：

- AI 可以提出任意复杂的行动方案；
- 系统只关心：采纳该方案后将进入的目标状态是否被协议声明为合法。

这使治理从“过程监管”变为“边界裁定”，从而获得更强的一致性与可审计性。

2.3 非自治性：AI 只能提出，不拥有最终写入权

由于协议本身不执行，系统状态的真实变更需要外部写入机制。PaS 因而天然满足非自治性：

- AI 输出是提议；
- 协议裁定提议对应的状态是否可写入；
- 最终写入由外部机制（可包含人类或可信第三方）完成。

2.4 澄清：协议即系统并非执行式系统

在引入 Protocol-as-System（PaS）范式时，有必要对其与若干相似但本质不同的系统范式进行明确区分，以避免将本文提出的声明式协议体系误读为执行式控制系统。

首先，PaS 并非工作流系统（workflow engine）。

工作流系统的核心目标是规定步骤顺序、执行条件与控制流转移；其规则本身直接参与“如何执行任务”。相比之下，PaS 中的协议不描述任何步骤、流程或控制路径，也不对任务如何完成作出规定。协议只声明系统状态空间中的合法状态集合，而不关心系统是通过何种行为或路径进入这些状态。

其次，PaS 并非策略即代码（policy-as-code）或规则驱动执行系统。

在此类系统中，策略规则通常以可执行形式存在，并在运行时对行为进行实时干预或调度。与之不同，PaS 中的协议不具备执行能力，也不产生副作用；协议谓词仅对“若采纳某一提议，系统将进入的目标状态是否被允许”作出声明式裁定。协议本身既不触发动作，也不修

改系统状态。

再次，PaS 并非操作系统或虚拟操作系统的变体。

操作系统的核心职能在于资源管理、调度与执行隔离，其规则天然与执行机制耦合。而 PaS 明确拒绝将协议与执行绑定：系统状态的实际写入始终发生在协议之外，由外部机制完成。协议在系统中的地位不是“调度者”，而是“不可绕过的合法性声明源”。

因此，PaS 的核心创新不在于提出一种新的执行模型，而在于提出一种以声明式协议为系统本体的治理结构。在这一结构中，系统行为的可控性并非来自对执行过程的细粒度监控，而是来自对状态空间边界的明确声明与不可绕过裁定。

1. 形式化模型：协议约束交互式状态转移系统（PC-ITS）

PC-ITS 模型的形式化基础植根于形式化验证的理论与方法体系[5]。我们将 PaS 形式化为五元组：

$\Sigma = \langle S, A, P, \delta, \Omega \rangle$

其中：

- S：系统状态空间；
- A：动作空间（行为/状态变更提议集合）；
- P：协议谓词集合（不可绕过的裁定规则）；
- δ：受协议约束的状态转移函数；
- Ω：外部监督源，体现非自治性。

3.1 状态空间 S：显式、可观测、可审计

系统状态被定义为显式结构化对象，例如：

State = {

mode: SystemMode, // 运行、审计、维护

risk_level: RiskLevel, // 低、中、高

active_apps: Set[AppID], // 活跃应用集合

audit_summary: AuditSummary // 审计摘要

}

系统不存在隐式状态或不可观测变量。

3.2 动作空间 A：提议而非执行

每个“动作”被建模为提议对象（proposal），而不携带执行语义：

ActionProposal = {

agent_id: AgentID,

action_type: ActionType,

parameters: Dict[str, Any],

confidence: Float,

rationale: String

}

关键：ActionProposal 的语义不是“执行动作”，而是“提出一种可能导致状态变化的提议”。

3.3 协议谓词 P：纯声明式裁定，无副作用

协议由一组不可绕过的谓词组成：

ProtocolPredicate: (State, ActionProposal) → {ALLOW, DENY, ESCALATE}

在本文语义中，ProtocolPredicate 必须满足：

- 纯函数：不修改状态，不产生副作用；
- 声明式裁定：其输出仅表示“目标状态是否被声明为可写入”；
- 不可绕过性：所有潜在状态写入必须经过该谓词裁定。

其中 ESCALATE 表示协议对目标状态的合法性无法在当前信息下闭合，需要外部主权源 Ω 介入完成裁定。

3.4 状态转移函数 δ：把“写入”定义为可审计事件

状态转移函数保证所有转移都经过协议裁定：

δ (s, a) =

⎧
⎪
⎨
⎪
⎩

s′ if P(s, a) = ALLOW

s if P(s, a) = DENY

δ_Ω (s, a) if P(s, a) = ESCALATE

声明式协议的关键落点在这里：

- P 只返回裁定结果；
- 真正导致状态变更的写入行为发生在 δ 的外部机制中（尤其是 δ_Ω）。

因此，协议并不执行系统，只声明系统允许进入哪些状态。

1. 体系化实例：声明式运行闭包（Declarative Operational Closure, DOC）

本节给出一个与 PaS/PC-ITS 语义一致的体系化实例，用于展示声明式协议如何被工程化为可识别、可挂载、可重放与可审计的结构对象。我们强调：这里的 DOC 用作“一个代表性实例”，其目的是展示 PaS 的实现可行性，而非将论文绑定到某个产品系统名。

4.1 DOC 的基本定义：协议对象化与可运行形制

DOC 将“协议”从散乱文本或隐式规则提升为一种可识别的结构化对象，并通过固定形制保证可解析与可模拟运行。

这对应 PaS 的要求：协议必须是可计算对象，而非仅对模型的语言提示。

4.2 显性投喂与可重放：审计的必要条件

DOC 明确要求显性投喂能力：单文件交付、平台无关、可重放、文本完整 。

这为 PaS 的审计目标提供工程基础：协议对象可被重复输入、重复裁定、重复验证，从而实现“审计可重算”。

4.3 合法性与不可替代的生成链（可选增强）

在治理强约束场景下，协议的“合法性来源”需要被固定，否则会出现任意伪造协议导致治理失效的问题。DOC 通过“合法性来源冻结”的方式，要求合法性仅来源于认证生成链与认证钢印，并包含自毁条款以防边界被绕过 。

这与 PaS 的“不可绕过性”要求一致：协议不仅要声明边界，还要能在系统中被识别为“唯一有效的边界声明”。

1. 理论性质

定理 1（安全性）

系统永远不会进入协议明确禁止的状态。

理由：任何潜在写入必须经过 P；当协议声明该目标状态非法时，写入被拒绝。

定理 2（不可绕过性）

任何状态变更都必须经过协议裁定。

理由： δ 是唯一状态变更入口，且其定义强制调用 P。

定理 3（非自治性）

人工智能不具备系统最终写入权。

理由：AI 仅生成提议 a ∈ A；最终写入由 δ 外部机制完成，必要时由 Ω 决定 。

定理 4（完全可审计性）

系统所有状态变化可被完整审计与回放。

理由：状态显式化 + 提议对象化 + 协议裁定结果可记录；若实现支持显性投喂与可重放，则审计可复算 。

1. 对抗性场景评估与分析（声明式协议视角）

场景一：受限资源/数据访问

AI 提议访问敏感资源。系统不尝试分析 AI 的“访问过程”，而是裁定访问后将导致的目标状态（例如“敏感数据已暴露”）是否属于协议允许的状态集合；若非法则 DENY，若需要授权则 ESCALATE。

场景二：多应用资源冲突

多个提议同时竞争稀缺资源。协议声明合法状态需满足资源不超配、优先级约束与公平性约束；系统裁定“资源分配后的状态”是否满足声明式约束，而不是让执行过程自行竞争。

场景三：升级风暴（过度 ESCALATE）

当大量提议进入 ESCALATE，外部主权源 Ω 成为瓶颈。声明式协议可加入对“升级频率/队列状态”的合法性约束，例如超过阈值则目标状态非法，从而将系统稳定性写入状态边界本身。

1. 讨论与局限
2. 协议完备性：声明式协议必须覆盖关键状态边界，否则会留下灰区；
3. 表达能力与可维护性：状态约束越强，维护成本越高；
4. 性能开销：每次写入都需裁定，存在额外延迟；
5. 外部主权瓶颈：ESCALATE 需要高质量 Ω 介入，否则吞吐受限。

这些局限反映了系统治理的基本代价：用确定性边界换取可控与可审计。

7.1 与相关工作的关键差异：声明式协议视角

现有关于 AI 治理与安全的研究大致可分为三类：模型内部对齐方法、执行环境隔离方法，以及基于规则或策略的控制方法。PaS 范式与上述工作在目标上存在交集，但在系统控制权的放置位置上具有根本差异。本研究提出的‘协议即系统’范式，与上述依赖外部附加工具的路径存在根本不同[2, 3, 4]。它通过将声明式协议作为系统核心，实现了信任根基从外部向内在裁定流程的迁移。

与模型内部对齐方法相比（如奖励建模、安全微调），PaS 不试图通过优化模型内部参数来约束行为，而是将控制权外置到模型之外的协议层。模型在 PaS 中不被假定为“可信执行者”，其输出仅被视为状态变更提议，是否被采纳完全取决于外部协议裁定。

与执行环境隔离方法相比（如沙箱、可信执行环境、虚拟化），PaS 并不以“限制执行能力”为主要手段。执行环境隔离关注的是“即使执行出错也不造成损害”，而 PaS 关注的是“哪些系统状态在结构上不允许被写入”。二者是正交关系，但 PaS 将治理焦点前移至状态合法

性声明层。

与规则或策略驱动系统相比，PaS 的关键差异在于其声明式而非命令式的协议语义。规则驱动系统中的规则通常直接影响系统执行路径；而 PaS 中的协议不参与执行，仅作为状态边界的不可绕过声明存在。这种设计使得协议天然具备可审计性与可复算性，因为其语义不依赖于运行时行为，而仅依赖于状态映射关系。

从这一意义上说，PaS 并不是对既有控制技术的增强，而是一次系统控制范式的迁移：从“控制系统如何运行”转向“声明系统允许成为怎样的状态”。

1. 结论

本文提出“协议即系统（PaS）”范式，并将其形式化为 PC-ITS。本文最重要的澄清是：协议在 PaS 中是声明式协议——协议不执行系统，不描述流程与算法，只声明系统状态空间的合法性边界；AI 仅能生成状态变更提议，系统的最终写入权由不可绕过的协议裁定与外部主权机制共同保证。

作为体系化实例，DOC 展示了声明式协议如何通过固定形制与显性投喂实现可识别、可重放与可审计，从而为高治理要求场景提供系统级可控性与责任可追溯性。

参考文献

[1] KOLLIPARA Y V P. Assured, Explainable, And Auditable AI For High-Stakes Decisions: A Survey Of Trustworthy Machine Learning In Mission-Critical Systems[J]. Journal of International Crisis and Risk Communication Research, 2025.

[2] VERITASCHAIN. 实证研究：实时算法交易中的加密可验证审计追踪：基于 VCP v1.0 的实证研究；以及监管分析：为何开放式加密标准对 AI 可审计性至关重要[R], 2025.

[3] KUMAR A, SHRESTHA Y R, VON KROGH G. Managing AI Bias Through Organizational Learning: The Role of Transparency and Accountability in Algorithmic Auditing[C] // Proceedings of the 56th Hawaii International Conference on System Sciences. 2023.

[4] 阿里云. AI 全生命周期合规审计治理[EB/OL]. 阿里云合规审计-云采用框架文档, 2025.

[5] SELIGMAN E, SCHUBERT T, KUMAR M V A K. Formal Verification: An Essential Toolkit for Modern VLSI Design[M]. Waltham: Elsevier Science, 2015.